



Skrive til Siemens PLC

Write commands med Python `snappy`

Laeringsmaal

- ✓ Skrive Bool, Int og Real til en Data Block via `db_write`
- ✓ Laese og skrive fysiske indgange (%I), udgange (%Q) og memory bits (%M) med `read_area` og `write_area`
- ✓ Forstaae forskellen paa `S7AreaPE`, `S7AreaPA` og `S7AreaMK`
- ✓ Implementere robust fejlhaandtering med `try/except/finally`



Indhold

1	Introduktion	2
2	Skriv til Data Block (db_write)	3
3	Fysiske indgange og udgange (%I, %Q, %M)	5
3.1	Laes fysisk indgang (%I)	5
3.2	Laes og skriv fysisk udgang (%Q)	6
3.3	Laes og skriv memory bits (%M)	7
4	Fejlhaandtering (try / except / finally)	8
5	Oversigt over write-funktioner	9
6	Oevelse	9

1 | Introduktion

I dag 04 lærte vi at læse data fra en Siemens PLC via Data Blocks. I dag 05 udvider vi med **write commands** — vi skriver værdier tilbage til PLC'en og arbejder desuden med PLC'ens fysiske I/O-områder (%I, %Q, %M) ved hjælp af `read_area` og `write_area`.

Laes foer du skriver

Naar du skriver til en Data Block, skal du **altid læse det aktuelle bytearray foerst**. Ellers risikerer du at overskrive bits i samme byte, der styrer andre variable.

Eksempel: Bool DBX0.0 og DBX0.1 deler samme byte (byte 0). Læs hele byten, ændr kun det ønskede bit, skriv byten tilbage.

2 | Skriv til Data Block (db_write)

Med `set_bool`, `set_int`, `set_real` modificerer vi et bytearray lokalt og sender det derefter til PLC'en med `db_write`.

Datatype	Offset	Stroerrelse	Funktion	TIA Portal navn
Bool	0	1 byte	<code>set_bool</code>	DB1.DBX0.0
Int	2	2 bytes	<code>set_int</code>	DB1.DBW2
Real	4	4 bytes	<code>set_real</code>	DB1.DBD4

 Kode 1 — Skriv Bool, Int og Real til DB1

```
1 import snap7
2 from snap7.util import get_bool, get_int, get_real
3
4 # ===== PLC opsætning =====
5 PLC_IP      = "192.168.0.1"
6 RACK        = 0
7 SLOT        = 1
8 DB_NUMBER   = 1
9 BOOL_OFFSET = 0; BOOL_SIZE = 1
10 INT_OFFSET  = 2; INT_SIZE  = 2
11 REAL_OFFSET = 4; REAL_SIZE = 4
12
13 # ===== PLC forbindelse =====
14 client = snap7.client.Client()
15 client.connect(PLC_IP, RACK, SLOT)
16
17 # ===== Læs input =====
18 raw_bool = client.db_read(DB_NUMBER, BOOL_OFFSET, BOOL_SIZE)
19 raw_int  = client.db_read(DB_NUMBER, INT_OFFSET, INT_SIZE)
20 raw_real = client.db_read(DB_NUMBER, REAL_OFFSET, REAL_SIZE)
21
22 # ===== Logik =====
23 set_bool(raw_bool, 0, 0, True) # sæt DBX0.0 til True
24 set_int(raw_int, 0, 123)      # sæt DBW2 til 123
25 set_real(raw_real, 0, 123.45) # sæt DBD4 til 123.45
26
27 # ===== Skriv output =====
28 client.db_write(DB_NUMBER, BOOL_OFFSET, raw_bool)
29 client.db_write(DB_NUMBER, INT_OFFSET, raw_int)
30 client.db_write(DB_NUMBER, REAL_OFFSET, raw_real)
31
32 print("Bool:", get_bool(client.db_read(DB_NUMBER, BOOL_OFFSET,
33   ↪   BOOL_SIZE), 0, 0))
34 print("Int:", get_int(client.db_read(DB_NUMBER, INT_OFFSET, INT_SIZE),
35   ↪   0))
36 print("Real:", round(get_real(client.db_read(DB_NUMBER, REAL_OFFSET,
37   ↪   REAL_SIZE), 0), 2))
38
39 # ===== Disconnect og destroy ==
40 client.disconnect()
41 client.destroy()
```

3 | Fysiske indgange og udgange (%I, %Q, %M)

Ud over Data Blocks kan vi med `read_area` og `write_area` tilgaa PLC'ens fysiske I/O-omraader direkte.

Omraade-konstanter i snap7

Konstant	Omraade	Eksempel
S7AreaPE	Fysiske indgange	%I0.0
S7AreaPA	Fysiske udgange	%Q0.0
S7AreaMK	Memory bits	%M0.0
S7AreaDB	Data Block	DB1

3.1 Laes fysisk indgang (%I)

Kode 2 — Laes fysisk indgang %I0.0

```

1 import snap7
2 from snap7.util import get_bool
3 from snap7.type import Area # PE = fysiske indgange
4
5 # ===== PLC opsætning =====
6 PLC_IP   = "192.168.0.1"
7 RACK     = 0
8 SLOT     = 1
9 AREA     = Area.PE
10 I_OFFSET = 0; I_SIZE = 1
11
12 # ===== PLC forbindelse =====
13 client = snap7.client.Client()
14 client.connect(PLC_IP, RACK, SLOT)
15
16 # ===== Laes input =====
17 i_bytes_raw = client.read_area(AREA, 0, I_OFFSET, I_SIZE)
18
19 # ===== Logik =====
20 S1 = get_bool(i_bytes_raw, 0, 0) # I0.0
21 print(f"S1 (I0.0): {S1}")
22
23 # ===== Disconnect og destroy ===
24 client.disconnect()
25 client.destroy()

```

3.2 Laes og skriv fysisk udgang (%Q)

Kode 3 — Skriv til fysisk udgang %Q0.0

```
1 import snap7
2 from snap7.util import get_bool, set_bool
3 from snap7.type import Area # PA = fysiske udgange
4
5 # ===== PLC opsætning =====
6 PLC_IP = "192.168.0.1"
7 RACK = 0
8 SLOT = 1
9 AREA = Area.PA
10 Q_OFFSET = 0; Q_SIZE = 1
11
12 # ===== PLC forbindelse =====
13 client = snap7.client.Client()
14 client.connect(PLC_IP, RACK, SLOT)
15
16 # ===== Laes input =====
17 q_bytes_raw = client.read_area(AREA, 0, Q_OFFSET, Q_SIZE)
18 Q1_foer = get_bool(q_bytes_raw, 0, 0)
19 print(f"Q1 foer: {Q1_foer}")
20
21 # ===== Logik =====
22 set_bool(q_bytes_raw, 0, 0, True) # saet Q0.0 til True
23
24 # ===== Skriv output =====
25 client.write_area(AREA, 0, Q_OFFSET, q_bytes_raw)
26 Q1_eter = get_bool(client.read_area(AREA, 0, Q_OFFSET, Q_SIZE), 0, 0)
27 print(f"Q1 eter: {Q1_eter}")
28
29 # ===== Disconnect og destroy ===
30 client.disconnect()
31 client.destroy()
```

3.3 Laes og skriv memory bits (%M)

Kode 4 — Laes og skriv memory bit %M0.0

```
1 import snap7
2 from snap7.util import get_bool, set_bool
3 from snap7.type import Area # MK = memory bits
4
5 # ===== PLC opsætning =====
6 PLC_IP = "192.168.0.1"
7 RACK = 0
8 SLOT = 1
9 AREA = Area.MK
10 M_OFFSET = 0; M_SIZE = 1
11
12 # ===== PLC forbindelse =====
13 client = snap7.client.Client()
14 client.connect(PLC_IP, RACK, SLOT)
15
16 # ===== Laes input =====
17 m_bytes_raw = client.read_area(AREA, 0, M_OFFSET, M_SIZE)
18 M0_foer = get_bool(m_bytes_raw, 0, 0)
19 print(f"M0.0 foer: {M0_foer}")
20
21 # ===== Logik =====
22 set_bool(m_bytes_raw, 0, 0, True) # saet M0.0 til True
23
24 # ===== Skriv output =====
25 client.write_area(AREA, 0, M_OFFSET, m_bytes_raw)
26 M0_efter = get_bool(client.read_area(AREA, 0, M_OFFSET, M_SIZE), 0, 0)
27 print(f"M0.0 efter: {M0_efter}")
28
29 # ===== Disconnect og destroy ===
30 client.disconnect()
31 client.destroy()
```


4 | Fejlhaandtering (try / except / finally)

Brug try/except/finally til at sikre at forbindelsen *altid* lukkes — ogsaa hvis der opstaaar en fejl undervejs. Med `client.get_connected()` kan vi tjekke forbindelsen foer vi kalder `disconnect`.

Kode 5 — Robust fejlhaandtering

```
1 import snap7
2 from snap7.util import get_bool, set_bool
3 from snap7.type import Area
4
5 # ===== PLC opsaetning =====
6 PLC_IP   = "192.168.0.1"
7 RACK     = 0
8 SLOT     = 1
9 AREA     = Area.PA
10 Q_OFFSET = 0; Q_SIZE = 1
11
12 try:
13     # ===== PLC forbindelse =====
14     client = snap7.client.Client()
15     client.connect(PLC_IP, RACK, SLOT)
16
17     # ===== Laes input =====
18     q_bytes_raw = client.read_area(AREA, 0, Q_OFFSET, Q_SIZE)
19
20     # ===== Logik =====
21     set_bool(q_bytes_raw, 0, 0, True)
22     print("Alt gik godt!")
23
24     # ===== Skriv output =====
25     client.write_area(AREA, 0, Q_OFFSET, q_bytes_raw)
26
27 except snap7.exceptions.Snap7Exception as e:
28     print(f"Snap7 fejl: {e}")
29 except Exception as e:
30     print(f"Uventet fejl: {e}")
31
32 finally:
33     # ===== Disconnect og destroy ===
34     if client.get_connected():
35         client.disconnect()
36         print("Forbindelsen er lukket.")
37     client.destroy()
38     print("Ressourcer frigivet.")
```

i try / except / finally forklaret

try	Kode der <i>kan</i> fejle koeres her.
except	Fanger specifikke fejl — <code>Snap7Exception</code> for PLC-fejl, <code>Exception</code> som fallback.
finally	Koeres altid uanset fejl. <code>get_connected()</code> bruges til at tjekke forbindelsen foer <code>disconnect</code> .

5 | Oversigt over write-funktioner

Funktion	Beskrivelse	Modul
<code>set_bool(data, byte, bit, val)</code>	Saetter en Bool i bytearray	<code>snap7.util</code>
<code>set_int(data, byte, val)</code>	Saetter en Int i bytearray	<code>snap7.util</code>
<code>set_real(data, byte, val)</code>	Saetter en Real i bytearray	<code>snap7.util</code>
<code>set_word(data, byte, val)</code>	Saetter en Word i bytearray	<code>snap7.util</code>
<code>client.db_write(db, off, data)</code>	Skriver bytearray til DB	<code>snap7</code>
<code>client.write_area(area, db, off, data)</code>	Skriver til I/O-omraade	<code>snap7</code>
<code>client.get_connected()</code>	Returnerer True hvis forbundet	<code>snap7</code>
<code>S7AreaPE</code>	Fysiske indgange (%I)	<code>snap7.type</code>
<code>S7AreaPA</code>	Fysiske udgange (%Q)	<code>snap7.type</code>
<code>S7AreaMK</code>	Memory bits (%M)	<code>snap7.type</code>

6 | Oevelse

☰ Oevelsesopgaver

1. Skriv en Bool, Int og Real til DB1 og laes dem tilbage for at verificere.
2. Laes den fysiske indgang I0.0 og udskriv dens tilstand.
3. Skriv til den fysiske udgang Q0.0 — taend og slaek den.
4. Laes og skriv en memory bit M0.0.
5. Omskriv et af eksemplerne med `try/except/finally` og brug `client.get_connected()` i `finally`-blokken.
6. **Ekstra:** Laes knap I0.0 og styr lampe Q0.0 baseret paa knappens tilstand (if/else).